



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

ODJEL ZA MATEMATIKU

Sveučilišni diplomski studij matematike
smjer: Matematika i računarstvo

Reed-Solomonovi kodovi za ispravljanje pogrešaka

SEMINARSKI RAD

Mentor:

Ime i prezime mentora

Kandidat:

Jurica Jurčević

Osijek, 2026.

Sadržaj

1	Uvod	1
2	Matematička podloga	3
2.1	Konačna polja	3
2.2	Polinomi nad poljem	4
3	Definicija kôda i kodiranje	5
3.1	Parametri kôda	5
3.2	Generirajući polinom	5
4	Dekodiranje	7
4.1	Sindromi	7
4.2	Polinom lokatora pogrešaka	7
4.3	Chienova pretraga	8
4.4	Forneyev algoritam	8
4.5	Primjer nad većim poljem	8
5	Primjene	9
6	Zaključak	11
	Literatura	13
	Sažetak	15
	Summary	17
	Životopis	19

1 | Uvod

Podatak koji putuje kroz stvarni kanal rijetko stigne netaknut. Grebotina na CD-u, oštećen dio QR koda, šum na radijskoj vezi, sve to mijenja pojedine bitove. Kodovi za ispravljanje pogrešaka rješavaju taj problem tako da uz podatak pošalju dodatne simbole iz kojih se izvorni podatak može rekonstruirati, čak i kad dio primljenog niza nije točan.

Reed-Solomonovi kodovi jedni su od najraširenijih takvih kodova. Irving Reed i Gustave Solomon opisali su ih 1960. godine u kratkom radu od pet stranica [1]. Ideja je algebarska. Podatak se shvati kao polinom nad konačnim poljem, a kodna riječ nastaje iz vrijednosti tog polinoma. Dvije razne poruke daju polinome koji se podudaraju u malom broju točaka, pa se i njihove kodne riječi razlikuju na dovoljno mjesta da se pogreške mogu i otkriti i ispraviti.

Snaga tih kodova je u radu sa simbolima, ne s pojedinačnim bitovima. Jedan simbol nad poljem $GF(2^8)$ nosi osam bitova, a kôd ga tretira kao jednu jedinicu. Oštećenje cijelog bajta broji se kao jedna pogreška, bez obzira na to koliko je bitova unutar njega promijenjeno. Zato su Reed-Solomonovi kodovi dobri protiv rafalnih pogrešaka, onih koje pogode niz uzastopnih bitova odjednom. Upravo takve nastaju kod grebotine na disku.

Rad kreće od potrebne algebre, konačnih polja i polinoma nad njima. Zatim definiira kôd i pokazuje kodiranje preko generirajućeg polinoma. Slijedi dekodiranje, koje je zahtjevniji dio, kroz sindrome, Berlekamp-Masseyev algoritam, Chienovu pretragu i Forneyev algoritam. Svaki korak prati potpuno proveden brojčani primjer nad malim poljem $GF(8)$. Na kraju su primjene i kratak osvrt na granice kôda. Svi brojčani rezultati u radu proizvedeni su priloženom implementacijom `kod/rs.py`.

2 | Matematička podloga

2.1 Konačna polja

Reed-Solomonovi kodovi rade nad konačnim poljem. Za svaki prosti broj p i prirodan broj m postoji polje s p^m elemenata, jedinstveno do na izomorfizam, koje označavamo s $GF(p^m)$. U praksi je gotovo uvijek $p = 2$, jer se tada jedan simbol poklapa s nizom bitova. Polje $GF(2^8)$ ima 256 elemenata i svaki od njih stane u jedan bajt.

Definicija 1. Polje $GF(2^m)$ konstruiramo kao skup polinoma nad $GF(2)$ stupnja manjeg od m , uz zbrajanje i množenje po modulu nekog nesvodljivog polinoma stupnja m .

Zbrajanje u $GF(2^m)$ je zbrajanje polinoma nad $GF(2)$, što se svodi na operaciju *isključivo* ili po bitovima. Množenje je množenje polinoma, a zatim uzimanje ostatka po nesvodljivom polinomu. Svaki element različit od nule može se zapisati kao potencija jednog jedinog elementa α , koji zovemo primitivnim. To svojstvo je ono što cijelu konstrukciju čini upotrebljivom.

Primjer 1. Uzmimo $GF(8)$ s nesvodljivim polinomom $x^3 + x + 1$. Primitivni element je $\alpha = x$, pa vrijedi $\alpha^3 = \alpha + 1$. Uzastopnim množenjem dobivamo cijelu tablicu polja:

potencija	polinom	bitovi
α^0	1	001
α^1	x	010
α^2	x^2	100
α^3	$x + 1$	011
α^4	$x^2 + x$	110
α^5	$x^2 + x + 1$	111
α^6	$x^2 + 1$	101

Tablica 2.1: Elementi polja $GF(8)$ kao potencije primitivnog elementa α .

Iz tablice se lako množi. Umnožak $\alpha^i \cdot \alpha^j$ je $\alpha^{(i+j) \bmod 7}$, jer je $\alpha^7 = 1$. Tako se množenje svodi na zbrajanje eksponenata, a dijeljenje na oduzimanje. Priložena implementacija upravo ovako radi, s tablicom logaritama i tablicom potencija.

2.2 Polinomi nad poljem

Poruku duljine k simbola shvaćamo kao polinom

$$m(x) = m_{k-1}x^{k-1} + \cdots + m_1x + m_0,$$

gdje su koeficijenti m_i elementi polja $GF(2^m)$. Kodiranje i dekodiranje su operacije nad takvim polinomima. Ključna činjenica koju koristimo je da polinom stupnja d nad poljem ima najviše d nultočaka. Iz toga slijedi razmak između kodnih riječi, a razmak je ono što omogućuje ispravljanje.

3 | Definicija kôda i kodiranje

3.1 Parametri kôda

Reed-Solomonov kôd označavamo s $RS(n, k)$. Ovdje je n duljina kodne riječi u simbolima, a k broj podatkovnih simbola. Za polje $GF(2^m)$ vrijedi $n = 2^m - 1$ za puni kôd, dok su kraće inačice dopuštene. Razlika $n - k$ je broj dodatnih, paritetnih simbola.

Teorem 1 (vidjeti [2, pogl. 5]). *Minimalna udaljenost Reed-Solomonovog kôda $RS(n, k)$ iznosi $d = n - k + 1$. Kôd time dostiže Singletonovu granicu $d \leq n - k + 1$ s jednakošću, pa je optimalan u smislu razmaka.*

Kôd s minimalnom udaljenošću d ispravlja do

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{n-k}{2} \right\rfloor$$

pogrešaka. Svaki dodatni paritetni simbol podiže sposobnost ispravljanja za pola pogreške. Dva paritetna simbola ispravljaju jednu pogrešku, deset ih ispravlja pet.

3.2 Generirajući polinom

Sistematsko kodiranje gradi kodnu riječ tako da podatak ostane vidljiv u njoj, a paritet se doda ispred. Temelji se na generirajućem polinomu

$$g(x) = \prod_{i=1}^{n-k} (x - \alpha^i), \quad (3.1)$$

čije su nultočke uzastopne potencije primitivnog elementa. Kodna riječ mora biti djeljiva s $g(x)$. Za poruku $m(x)$ računamo

$$c(x) = x^{n-k}m(x) - (x^{n-k}m(x) \bmod g(x)). \quad (3.2)$$

Prvi član pomiče podatak za $n - k$ mjesta i ostavlja prazan prostor za paritet. Drugi član je ostatak dijeljenja, koji se oduzme kako bi razlika bila djeljiva s $g(x)$. U polju karakteristike 2 oduzimanje i zbrajanje su ista operacija.

Primjer 2. Za $RS(7, 3)$ nad $GF(8)$ imamo $n - k = 4$ paritetna simbola, pa kôd ispravlja $t = 2$ pogreške. Generirajući polinom prema (3.1) ispada

$$g(x) = x^4 + \alpha^3 x^3 + x^2 + \alpha^1 x + \alpha^3.$$

Uzmimo poruku $m(x) = x^2$. Formula (3.2) daje kodnu riječ

$$c(x) = x^6 + \alpha^4 x^3 + x^2 + \alpha^4 x + \alpha^5.$$

Provjera potvrđuje da je $c(\alpha^j) = 0$ za $j = 1, 2, 3, 4$, dakle kodna riječ je djeljiva s $g(x)$.

4 | Dekodiranje

Dekodiranje je teži smjer. Primatelj dobiva riječ $r(x) = c(x) + e(x)$, gdje je $e(x)$ nepoznati polinom pogreške. Treba pronaći gdje su pogreške i kolike su, a da se ne zna ni koliko ih ima. Postupak ide u četiri koraka.

4.1 Sindromi

Sindrom je vrijednost primljene riječi u jednoj nultočki generirajućeg polinoma:

$$S_j = r(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j), \quad j = 1, \dots, n - k. \quad (4.1)$$

Kako je $c(\alpha^j) = 0$, sindrom ovisi samo o pogrešci. Ako su svi sindromi nula, primljena riječ je ispravna kodna riječ i dekodiranje staje. Inače sindromi nose podatke o pogreškama, ali u zamotanom obliku koji tek treba razriješiti.

Primjer 3. Nastavljamo primjer 2. Neka se kodna riječ ošteti na dva mjesta, s pogreškom α^2 na poziciji x^5 i pogreškom α^3 na poziciji x^2 . Primljena riječ je

$$r(x) = x^6 + \alpha^2 x^5 + \alpha^4 x^3 + \alpha^1 x^2 + \alpha^4 x + \alpha^5.$$

Uvrštavanjem prema (4.1) dobivamo sindrome

$$(S_1, S_2, S_3, S_4) = (\alpha^4, \alpha^4, \alpha^5, \alpha^2).$$

Nisu svi nula, pa riječ sadrži pogreške.

4.2 Polinom lokatora pogrešaka

Neka su pogreške na pozicijama iz skupa indeksa. Za svaku pogrešku definiramo lokator $X_l = \alpha^{i_l}$, gdje je i_l pozicija. Polinom lokatora pogrešaka je

$$\Lambda(x) = \prod_l (1 - X_l x),$$

a njegove nultočke su inverzi lokatora. Sindromi i koeficijenti od $\Lambda(x)$ povezani su sustavom linearnih jednadžbi. Taj sustav rješava Berlekamp-Masseyev algoritam, koji gradi $\Lambda(x)$ postupno, dodajući jedan sindrom po koraku i ispravljajući polinom kad se ne slaže s idućim sindromom. Algoritam usput otkriva i broj pogrešaka, što unaprijed nismo znali.

Primjer 4. Za sindrome iz primjera 3 Berlekamp-Masseyev algoritam vraća

$$\Lambda(x) = x^2 + \alpha^3 x + 1,$$

polinom stupnja 2, što znači dvije pogreške. To se slaže s onim što smo ubacili.

4.3 Chienova pretraga

Nultočke od $\Lambda(x)$ nalazimo iscrpnom provjerom svih elemenata polja. To je Chienova pretraga. Za svaku poziciju i provjerimo je li $\Lambda(\alpha^{-i}) = 0$. Ako jest, na toj poziciji je pogreška. Polje je konačno, pa je pretraga uvijek izvediva, a u sklopovlju se izvodi vrlo brzo.

Primjer 5. Pretraga nad $\Lambda(x)$ iz primjera 4 pronalazi nultočke koje odgovaraju pozicijama x^2 i x^5 . To su točno mjesta na kojima smo unijeli pogreške.

4.4 Forneyev algoritam

Ostaje odrediti kolike su pogreške. Forneyev algoritam daje vrijednost pogreške na svakoj pronađenoj poziciji preko formule koja koristi polinom sindroma, polinom lokatora i njegovu formalnu derivaciju. Definiramo evaluator pogreške

$$\Omega(x) = S(x) \Lambda(x) \bmod x^{n-k},$$

a vrijednost pogreške na poziciji s lokatorom X_l je

$$e_l = \frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}.$$

Kad znamo i pozicije i vrijednosti, pogrešku oduzmemo od primljene riječi i dobijemo natrag kodnu riječ.

Primjer 6. Za naš primjer evaluator ispada $\Omega(x) = \alpha^5 x + \alpha^4$. Forneyeva formula na pozicijama x^2 i x^5 daje vrijednosti α^3 odnosno α^2 . To su upravo unesene pogreške. Oduzimanjem od $r(x)$ vraćamo se na

$$c(x) = x^6 + \alpha^4 x^3 + x^2 + \alpha^4 x + \alpha^5,$$

izvornu kodnu riječ iz primjera 2. Ispravak je potpun.

4.5 Primjer nad većim poljem

Mali primjer nad $GF(8)$ pokazuje mehaniku, ali stvarni kodovi rade nad $GF(256)$. Priložena implementacija kodira 17 podatkovnih simbola s 10 paritetnih, pa ispravlja do 5 pogrešaka. Poruka je niz znakova Reed-Solomon 1960. Nakon što se ošteti pet simbola na različitim pozicijama, dekodir pronalazi svih pet i vraća izvornu poruku bez greške.

Granica ispravlivosti je oštra. Uz 10 paritetnih simbola šest pogrešaka je previše, i implementacija to prijavljuje umjesto da vrati krivi rezultat. Ovo je važno svojstvo. Kôd zna kad je premašen, pa se pogrešan ispravak može izbjeći.

5 | Primjene

Reed-Solomonovi kodovi su posvuda gdje podaci prolaze kroz nepouzdan kanal ili medij.

Na CD-u i DVD-u koristi se kombinacija dvaju isprepletenih Reed-Solomonovih kodova, poznata kao CIRC. Isprepletanje razbija dugu grebotinu na mnogo kratkih pogrešaka koje pojedini kôd lako ispravlja. Zahvaljujući tome disk s vidljivim ogrebotinama i dalje svira bez preskakanja.

QR kodovi ugrađuju Reed-Solomonov paritet u samu sliku. Ovisno o razini zaštite, ispravlja se od 7 do 30 posto oštećenih simbola. Zato QR kôd radi i kad je dio prekriven logotipom ili zamrljan.

RAID-6 sustavi za pohranu koriste Reed-Solomonovu aritmetiku za drugi paritetni disk. Time podnose ispad dvaju diskova istovremeno, bez gubitka podataka.

U svemirskim misijama kôd je bio dio komunikacije sonde Voyager, gdje se slabi signal s ruba Sunčeva sustava ispravlja na Zemlji. Slična shema živi u standardima digitalne televizije DVB i u dubinskom svemirskom komuniciranju [3].

Zajedničko svim tim primjenama je rad sa simbolima. Rafalna pogreška koja bi kôd nad pojedinačnim bitovima preplavila ovdje pogađa mali broj simbola, i kôd je ispravi.

6 | Zaključak

Reed-Solomonovi kodovi spajaju čistu algebru s vrlo praktičnim rezultatom. Poruka postaje polinom nad konačnim poljem, paritet nastaje iz djeljivosti generirajućim polinomom, a dekodiranje kroz četiri koraka vraća izvorne podatke i onda kad je dio primljenog niza oštećen. Proveden primjer nad $GF(8)$ pokazao je cijeli put, od kodiranja preko sindroma i lokatora do konačnog ispravka dvije pogreške.

Snaga kôda leži u radu sa simbolima i u dostizanju Singletonove granice, čime je optimalan u smislu razmaka među kodnim riječima. Granica ispravlјivosti je poznata unaprijed i kôd zna kad je premašen. Ta kombinacija pouzdanosti i predvidivosti objašnjava zašto je kôd iz 1960. godine i danas ugrađen u diskove, QR kodove, sustave pohrane i svemirske veze.

Literatura

- [1] I. S. REED, G. SOLOMON, *Polynomial Codes over Certain Finite Fields*, Journal of the Society for Industrial and Applied Mathematics **8**(1960), 300–304.
- [2] F. J. MACWILLIAMS, N. J. A. SLOANE, *The Theory of Error-Correcting Codes*, North-Holland, Amsterdam, 1977.
- [3] S. B. WICKER, V. K. BHARGAVA (UR.), *Reed-Solomon Codes and Their Applications*, IEEE Press, New York, 1994.
- [4] R. LIDL, H. NIEDERREITER, *Introduction to Finite Fields and Their Applications*, Cambridge University Press, Cambridge, 1994.
- [5] W. A. GEISEL, *Tutorial on Reed-Solomon Error Correction Coding*, NASA Technical Memorandum 102162, 1990.

Sažetak

Reed-Solomonovi kodovi ispravljaju pogreške nastale prijenosom podataka kroz nepouz dane kanale. Rad izlaže potrebnu algebru konačnih polja $GF(2^m)$ i polinoma nad njima, definira kôd $RS(n, k)$ i njegovu minimalnu udaljenost, te opisuje sistematsko kodiranje generirajućim polinomom. Dekodiranje je prikazano kroz sindrome, Berlekamp-Masseyev algoritam, Chienovu pretragu i Forneyev algoritam, uz potpuno proveden brojčani primjer nad poljem $GF(8)$ koji ispravlja dvije pogreške. Na kraju su navedene primjene u optičkim medijima, QR kodovima, sustavima pohrane i svemirskim komunikacijama.

Ključne riječi

reed-solomonovi kodovi, ispravljanje pogrešaka, konačna polja, generirajući polinom, berlekamp-massey, kodiranje kanala

Naslov rada na engleskom jeziku

Summary

Reed-Solomon codes correct errors introduced when data passes through unreliable channels. This paper presents the required algebra of finite fields $GF(2^m)$ and polynomials over them, defines the $RS(n, k)$ code and its minimum distance, and describes systematic encoding by a generator polynomial. Decoding is shown through syndromes, the Berlekamp-Massey algorithm, the Chien search, and the Forney algorithm, with a fully worked numerical example over $GF(8)$ that corrects two errors. Applications in optical media, QR codes, storage systems, and space communication close the paper.

Keywords

reed-solomon codes, error correction, finite fields, generator polynomial, berlekamp-massey, channel coding

Životopis

Ovdje pišem životopis u prvom licu jednine.